NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
adi@adi:~/PES2UG24CS052-PES-VCS$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
adi@adi:~/PES2UG24CS052-PES-VCS$
```

Screenshot 1A above

Screenshot 1B below

```
All Phase 1 tests passed.
adi@adi:~/PES2UG24CS052-PES-VCS$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
adi@adi:~/PES2UG24CS052-PES-VCS$
```

Object.c

```
GNU nano 7.2                                   object.c
// object.c — Content-addressable object store

#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <openssl/evp.h>

// ── PROVIDED ──────────────────────────────────────────────────────────

void hash_to_hex(const ObjectID *id, char *hex_out) {
    for (int i = 0; i < HASH_SIZE; i++) {
        sprintf(hex_out + i * 2, "%02x", id->hash[i]);
    }
    hex_out[HASH_HEX_SIZE] = '\0';
}

int hex_to_hash(const char *hex, ObjectID *id_out) {
    if (strlen(hex) < HASH_HEX_SIZE) return -1;
    for (int i = 0; i < HASH_SIZE; i++) {
        unsigned int byte;
        if (sscanf(hex + i * 2, "%2x", &byte) != 1) return -1;
        id_out->hash[i] = (uint8_t)byte;
    }
    return 0;
}

void compute_hash(const void *data, size_t len, ObjectID *id_out) {
    unsigned int hash_len;
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo     M-6 Copy
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
  GNU nano 7.2                                                    object.c
}

void compute_hash(const void *data, size_t len, ObjectID *id_out) {
    unsigned int hash_len;
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    EVP_DigestInit_ex(ctx, EVP_sha256(), NULL);
    EVP_DigestUpdate(ctx, data, len);
    EVP_DigestFinal_ex(ctx, id_out->hash, &hash_len);
    EVP_MD_CTX_free(ctx);
}

void object_path(const ObjectID *id, char *path_out, size_t path_size) {
    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(id, hex);
    snprintf(path_out, path_size, "%s/%.2s/%s", OBJECTS_DIR, hex, hex + 2);
}

int object_exists(const ObjectID *id) {
    char path[512];
    object_path(id, path, sizeof(path));
    return access(path, F_OK) == 0;
}

// ── IMPLEMENTATION ──────────────────────────────────────────────────

int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out) {
    char type_str[10];

    if (type == OBJ_BLOB) strcpy(type_str, "blob");
    else if (type == OBJ_TREE) strcpy(type_str, "tree");
    else if (type == OBJ_COMMIT) strcpy(type_str, "commit");
    else return -1;

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

```
  GNU nano 7.2                                                    object.c

    // 1. Header
    char header[64];
    int header_len = snprintf(header, sizeof(header), "%s %zu", type_str, len) + 1;

    // 2. Combine header + data
    size_t total_size = header_len + len;
    unsigned char *buffer = malloc(total_size);
    if (!buffer) return -1;

    memcpy(buffer, header, header_len);
    memcpy(buffer + header_len, data, len);

    // 3. Compute hash
    compute_hash(buffer, total_size, id_out);

    // 4. Deduplication
    if (object_exists(id_out)) {
        free(buffer);
        return 0;
    }

    // 5. Get final path
    char path[512];
    object_path(id_out, path, sizeof(path));

    // Extract directory path
    char dir[512];
    strcpy(dir, path);
    char *slash = strrchr(dir, '/');
    if (!slash) {
        free(buffer);
        return -1;

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
    }
    *slash = '\0';

    mkdir(dir, 0755);

    // 6. Temp file
    char temp_path[512];
    snprintf(temp_path, sizeof(temp_path), "%s.tmp", path);

    int fd = open(temp_path, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) {
        free(buffer);
        return -1;
    }

    // 7. Write data
    if (write(fd, buffer, total_size) != (ssize_t)total_size) {
        close(fd);
        free(buffer);
        return -1;
    }

    // 8. fsync
    fsync(fd);
    close(fd);

    // 9. Atomic rename
    if (rename(temp_path, path) != 0) {
        free(buffer);
        return -1;
    }

    free(buffer);

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

```
  GNU nano 7.2                                    object.c
    return 0;
}

int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out) {
    char path[512];
    object_path(id, path, sizeof(path));

    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    fseek(f, 0, SEEK_END);
    size_t size = ftell(f);
    rewind(f);

    unsigned char *buffer = malloc(size);
    if (!buffer) {
        fclose(f);
        return -1;
    }

    fread(buffer, 1, size, f);
    fclose(f);

    // Verify integrity
    ObjectID check;
    compute_hash(buffer, size, &check);

    if (memcmp(check.hash, id->hash, HASH_SIZE) != 0) {
        free(buffer);
        return -1;
    }

    // Parse header

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
GNU nano 7.2                                    object.c
    char *header_end = memchr(buffer, '\0', size);
    if (!header_end) {
        free(buffer);
        return -1;
    }

    char type_str[10];
    size_t data_size;

    sscanf((char *)buffer, "%s %zu", type_str, &data_size);

    if (strcmp(type_str, "blob") == 0) *type_out = OBJ_BLOB;
    else if (strcmp(type_str, "tree") == 0) *type_out = OBJ_TREE;
    else if (strcmp(type_str, "commit") == 0) *type_out = OBJ_COMMIT;
    else {
        free(buffer);
        return -1;
    }

    unsigned char *data_start = (unsigned char *)header_end + 1;

    *data_out = malloc(data_size);
    if (!*data_out) {
        free(buffer);
        return -1;
    }

    memcpy(*data_out, data_start, data_size);
    *len_out = data_size;

    free(buffer);
    return 0;
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy
```

Phase 2

2A

```
adi@adi:~/PES2UG24CS052-PES-VCS$ gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
adi@adi:~/PES2UG24CS052-PES-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
adi@adi:~/PES2UG24CS052-PES-VCS$
```

2B

```
adi@adi:~/PES2UG24CS052-PES-VCS$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
adi@adi:~/PES2UG24CS052-PES-VCS$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
adi@adi:~/PES2UG24CS052-PES-VCS$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
adi@adi:~/PES2UG24CS052-PES-VCS$
```

Code

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

Tree.c

```
  GNU nano 7.2                                                  tree.c
// tree.c — Tree object serialization and construction
//
// PROVIDED functions: get_file_mode, tree_parse, tree_serialize
// TODO functions:     tree_from_index
//
// Binary tree format (per entry, concatenated with no separators):
//   "<mode-as-ascii-octal> <name>\0<32-byte-binary-hash>"

#include "tree.h"
#include "index.h"
#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

// ── Mode Constants ──────────────────────────────────────────────────

#define MODE_FILE      0100644
#define MODE_EXEC      0100755
#define MODE_DIR       0040000

// ── PROVIDED ────────────────────────────────────────────────────────

uint32_t get_file_mode(const char *path) {
    struct stat st;
    if (lstat(path, &st) != 0) return 0;
    if (S_ISDIR(st.st_mode))  return MODE_DIR;
    if (st.st_mode & S_IXUSR) return MODE_EXEC;
    return MODE_FILE;
}

                                                  [ Read 203 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

```
  GNU nano 7.2                                                  tree.c
int tree_parse(const void *data, size_t len, Tree *tree_out) {
    tree_out->count = 0;
    const uint8_t *ptr = (const uint8_t *)data;
    const uint8_t *end = ptr + len;

    while (ptr < end && tree_out->count < MAX_TREE_ENTRIES) {
        TreeEntry *entry = &tree_out->entries[tree_out->count];

        const uint8_t *space = memchr(ptr, ' ', end - ptr);
        if (!space) return -1;

        char mode_str[16] = {0};
        size_t mode_len = space - ptr;
        if (mode_len >= sizeof(mode_str)) return -1;
        memcpy(mode_str, ptr, mode_len);
        entry->mode = strtol(mode_str, NULL, 8);

        ptr = space + 1;

        const uint8_t *null_byte = memchr(ptr, '\0', end - ptr);
        if (!null_byte) return -1;

        size_t name_len = null_byte - ptr;
        if (name_len >= sizeof(entry->name)) return -1;
        memcpy(entry->name, ptr, name_len);
        entry->name[name_len] = '\0';

        ptr = null_byte + 1;

        if (ptr + HASH_SIZE > end) return -1;
        memcpy(entry->hash.hash, ptr, HASH_SIZE);
        ptr += HASH_SIZE;

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
GNU nano 7.2                                                    tree.c
        tree_out->count++;
    }
    return 0;
}

static int compare_tree_entries(const void *a, const void *b) {
    return strcmp(((const TreeEntry *)a)->name, ((const TreeEntry *)b)->name);
}

int tree_serialize(const Tree *tree, void **data_out, size_t *len_out) {
    size_t max_size = tree->count * 296;
    uint8_t *buffer = malloc(max_size);
    if (!buffer) return -1;

    Tree sorted_tree = *tree;
    qsort(sorted_tree.entries, sorted_tree.count, sizeof(TreeEntry), compare_tree_entries);

    size_t offset = 0;
    for (int i = 0; i < sorted_tree.count; i++) {
        const TreeEntry *entry = &sorted_tree.entries[i];
        int written = sprintf((char *)buffer + offset, "%o %s", entry->mode, entry->name);
        offset += written + 1;
        memcpy(buffer + offset, entry->hash.hash, HASH_SIZE);
        offset += HASH_SIZE;
    }

    *data_out = buffer;
    *len_out = offset;
    return 0;
}

// ── IMPLEMENTATION ──────────────────────────────────────────────────────

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo     M-6 Copy
```

```
static int write_tree_level(const IndexEntry *entries, int count,
                            const char *prefix, ObjectID *id_out) {
    Tree tree;
    tree.count = 0;

    int i = 0;
    while (i < count) {
        // Each entry's path relative to this level is:
        //   full_path + strlen(prefix)
        const char *rel = entries[i].path + strlen(prefix);

        // Does a '/' appear in the remainder?  If so, this entry lives in a
        // subdirectory at this level.
        const char *slash = strchr(rel, '/');

        if (!slash) {
            // ── Plain file entry ──────────────────────────────────
            // rel is just "filename" with no further nesting.
            TreeEntry *te = &tree.entries[tree.count++];
            te->mode = entries[i].mode;
            te->hash = entries[i].hash;
            strncpy(te->name, rel, sizeof(te->name) - 1);
            te->name[sizeof(te->name) - 1] = '\0';
            i++;
        } else {
            // ── Subdirectory entry ────────────────────────────────
            // Extract the immediate child directory name, e.g. "src" from
            // "src/main.c".
            size_t dir_name_len = slash - rel;
            char dir_name[256];
            if (dir_name_len >= sizeof(dir_name)) return -1;
            memcpy(dir_name, rel, dir_name_len);
            dir_name[dir_name_len] = '\0';

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo     M-6 Copy
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
        // Build the new prefix for the recursive call, e.g. "src/"
        char sub_prefix[512];
        snprintf(sub_prefix, sizeof(sub_prefix), "%s%s/", prefix, dir_name);
        size_t sub_prefix_len = strlen(sub_prefix);

        // Collect all entries that start with sub_prefix (i.e., belong to
        // this subdirectory).
        int j = i;
        while (j < count &&
               strncmp(entries[j].path, sub_prefix, sub_prefix_len) == 0) {
            j++;
        }
        // entries[i..j-1] all live under sub_prefix.

        // Recurse to build the subtree.
        ObjectID sub_id;
        if (write_tree_level(entries + i, j - i, sub_prefix, &sub_id) != 0)
            return -1;

        // Add a directory entry pointing at the subtree.
        TreeEntry *te = &tree.entries[tree.count++];
        te->mode = MODE_DIR;
        te->hash = sub_id;
        strncpy(te->name, dir_name, sizeof(te->name) - 1);
        te->name[sizeof(te->name) - 1] = '\0';

        i = j; // skip past all entries we just handled
    }
}

// Serialize and write this tree level to the object store.
void   *data = NULL;
size_t  data_len = 0;

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

```
    if (tree_serialize(&tree, &data, &data_len) != 0) return -1;

    int ret = object_write(OBJ_TREE, data, data_len, id_out);
    free(data);
    return ret;
}

// Comparison function for sorting IndexEntry pointers by path (for qsort).
static int compare_index_entries(const void *a, const void *b) {
    return strcmp(((const IndexEntry *)a)->path,
                  ((const IndexEntry *)b)->path);
}

// Build a tree hierarchy from the current index and write all tree objects to
// the object store.  Returns the root tree's ObjectID in *id_out.
int tree_from_index(ObjectID *id_out) {
    Index index;
    if (index_load(&index) != 0) return -1;
    if (index.count == 0) return -1; // nothing staged

    // Sort entries by path so that sibling files and directories are adjacent.
    // write_tree_level relies on this ordering to group subdirectory entries.
    qsort(index.entries, index.count, sizeof(IndexEntry), compare_index_entries);

    // Kick off the recursive build from the root (empty prefix).
    return write_tree_level(index.entries, index.count, "", id_out);
}

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Task 3

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

3A

```
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
make: `pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  Makefile
  untracked:  tree.h
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  README.md
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  object.h
  untracked:  commit.h
  untracked:  tree.c

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776342383 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776342383 6 file2.txt
cat: #: No such file or directory
cat: Human-readable: No such file or directory
cat: text: No such file or directory
cat: format: No such file or directory
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS %
```

3B

```
cat: format: No such file or directory
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776342383 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776342383 6 file2.txt
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS %
```

Index.c

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
// index.c — Staging area implementation
//
// Text format of .pes/index (one entry per line, sorted by path):
//
//   <mode-octal> <64-char-hex-hash> <mtime-seconds> <size> <path>
//
// Example:
//   100644 a1b2c3d4e5f6...  1699900000 42 README.md
//   100644 f7e8d9c0b1a2...  1699900100 128 src/main.c
//
// This is intentionally a simple text format. No magic numbers, no
// binary parsing. The focus is on the staging area CONCEPT (tracking
// what will go into the next commit) and ATOMIC WRITES (temp+rename).
//
// PROVIDED functions: index_find, index_remove, index_status
// TODO functions:     index_load, index_save, index_add

#include "index.h"
#include "object.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <time.h>

static int compare_index_entries(const void *a, const void *b) {
    return strcmp(((const IndexEntry *)a)->path, ((const IndexEntry *)b)->path);
}

// ——— PROVIDED ———————————————————————————————————————————————————————————————————————————————

// Find an index entry by path (linear scan).
IndexEntry* index_find(Index *index, const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path, path) == 0)
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
        return &index->entries[i];
    }
    return NULL;
}

// Remove a file from the index.
// Returns 0 on success, -1 if path not in index.
int index_remove(Index *index, const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path, path) == 0) {
            int remaining = index->count - i - 1;
            if (remaining > 0)
                memmove(&index->entries[i], &index->entries[i + 1],
                        remaining * sizeof(IndexEntry));
            index->count--;
            return index_save(index);
        }
    }
    fprintf(stderr, "error: '%s' is not in the index\n", path);
    return -1;
}

// Print the status of the working directory.
//
// Identifies files that are staged, unstaged (modified/deleted in working dir),
// and untracked (present in working dir but not in index).
// Returns 0.
int index_status(const Index *index) {
    printf("Staged changes:\n");
    int staged_count = 0;
    // Note: A true Git implementation deeply diffs against the HEAD tree here.
    // For this lab, displaying indexed files represents the staging intent.
    for (int i = 0; i < index->count; i++) {
        printf("  staged:     %s\n", index->entries[i].path);
        staged_count++;
    }
    if (staged_count == 0) printf("  (nothing to show)\n");
    printf("\n");
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
printf("Unstaged changes:\n");
int unstaged_count = 0;
for (int i = 0; i < index->count; i++) {
    struct stat st;
    if (stat(index->entries[i].path, &st) != 0) {
        printf("  deleted:    %s\n", index->entries[i].path);
        unstaged_count++;
    } else {
        // Fast diff: check metadata instead of re-hashing file content
        if (st.st_mtime != (time_t)index->entries[i].mtime_sec || st.st_size != (off_t)index->entries[i].size) {
            printf("  modified:   %s\n", index->entries[i].path);
            unstaged_count++;
        }
    }
}
if (unstaged_count == 0) printf("  (nothing to show)\n");
printf("\n");

printf("Untracked files:\n");
int untracked_count = 0;
DIR *dir = opendir(".");
if (dir) {
    struct dirent *ent;
    while ((ent = readdir(dir)) != NULL) {
        // Skip hidden directories, parent directories, and build artifacts
        if (strcmp(ent->d_name, ".") == 0 || strcmp(ent->d_name, "..") == 0) continue;
        if (strcmp(ent->d_name, ".pes") == 0) continue;
        if (strcmp(ent->d_name, "pes") == 0) continue; // compiled executable
        if (strstr(ent->d_name, ".o") != NULL) continue; // object files

        // Check if file is tracked in the index
        int is_tracked = 0;
        for (int i = 0; i < index->count; i++) {
            if (strcmp(index->entries[i].path, ent->d_name) == 0) {
                is_tracked = 1;
                break;
            }
        }
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
            if (!is_tracked) {
                struct stat st;
                stat(ent->d_name, &st);
                if (S_ISREG(st.st_mode)) { // Only list regular files for simplicity
                    printf("  untracked:  %s\n", ent->d_name);
                    untracked_count++;
                }
            }
        }
        closedir(dir);
    }
    if (untracked_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    return 0;
}

// ——— TODO: Implement these ———————————————————————————————————————————————

// Load the index from .pes/index.
//
// HINTS - Useful functions:
//   - fopen (with "r"), fscanf, fclose : reading the text file line by line
//   - hex_to_hash                      : converting the parsed string to ObjectID
//
// Returns 0 on success, -1 on error.
int index_load(Index *index) {
    index->count = 0;

    FILE *f = fopen(INDEX_FILE, "r");
    if (!f) return 0;

    char line[1024];
    while (fgets(line, sizeof(line), f)) {
        if (index->count >= MAX_INDEX_ENTRIES) {
            fclose(f);
            return -1;
        }
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
        IndexEntry *entry = &index->entries[index->count];
        char hex[HASH_HEX_SIZE + 1];

        if (sscanf(line, "%o %64s %llu %u  %511[^\n]",
                &entry->mode, hex,
                (unsigned long long *)&entry->mtime_sec,
                &entry->size, entry->path) != 5) {
            fclose(f);
            return -1;
        }

        if (hex_to_hash(hex, &entry->hash) != 0) {
            fclose(f);
            return -1;
        }

        index->count++;
    }

    fclose(f);
    return 0;
}

// Save the index to .pes/index atomically.
//
// HINTS - Useful functions and syscalls:
//   - qsort                          : sorting the entries array by path
//   - fopen (with "w"), fprintf      : writing to the temporary file
//   - hash_to_hex                    : converting ObjectID for text output
//   - fflush, fileno, fsync, fclose  : flushing userspace buffers and syncing to disk
//   - rename                         : atomically moving the temp file over the old index
//
// Returns 0 on success, -1 on error.
int index_save(const Index *index) {
    Index sorted = *index;
    qsort(sorted.entries, sorted.count, sizeof(IndexEntry), compare_index_entries);

    char tmp_path[512];
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
    snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", INDEX_FILE);

    FILE *f = fopen(tmp_path, "w");
    if (!f) return -1;

    for (int i = 0; i < sorted.count; i++) {
        char hex[HASH_HEX_SIZE + 1];
        hash_to_hex(&sorted.entries[i].hash, hex);
        if (fprintf(f, "%o %s %llu %u %s\n",
                    sorted.entries[i].mode,
                    hex,
                    (unsigned long long)sorted.entries[i].mtime_sec,
                    sorted.entries[i].size,
                    sorted.entries[i].path) < 0) {
            fclose(f);
            return -1;
        }
    }

    if (fflush(f) != 0) {
        fclose(f);
        return -1;
    }
    if (fsync(fileno(f)) != 0) {
        fclose(f);
        return -1;
    }
    if (fclose(f) != 0) return -1;

    return rename(tmp_path, INDEX_FILE);
}

// Stage a file for the next commit.
//
// HINTS - Useful functions and syscalls:
//   - fopen, fread, fclose            : reading the target file's contents
//   - object_write                    : saving the contents as OBJ_BLOB
//   - stat / lstat                    : getting file metadata (size, mtime, mode)
//   - index_find                      : checking if the file is already staged
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
//
// Returns 0 on success, -1 on error.
int index_add(Index *index, const char *path) {
    struct stat st;
    if (stat(path, &st) != 0) return -1;
    if (!S_ISREG(st.st_mode)) return -1;

    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    if (st.st_size < 0 || (unsigned long long)st.st_size > SIZE_MAX) {
        fclose(f);
        return -1;
    }

    size_t len = (size_t)st.st_size;
    void *data = malloc(len);
    if (!data) {
        fclose(f);
        return -1;
    }

    if (len > 0 && fread(data, 1, len, f) != len) {
        free(data);
        fclose(f);
        return -1;
    }
    fclose(f);

    ObjectID blob_id;
    if (object_write(OBJ_BLOB, data, len, &blob_id) != 0) {
        free(data);
        return -1;
    }
    free(data);

    IndexEntry *entry = index_find(index, path);
    if (!entry) {
        if (index->count >= MAX_INDEX_ENTRIES) return -1;
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
        entry = &index->entries[index->count++];
    }

    entry->mode = (st.st_mode & S_IXUSR) ? 0100755 : 0100644;
    entry->hash = blob_id;
    entry->mtime_sec = (uint64_t)st.st_mtime;
    entry->size = (uint32_t)st.st_size;
    snprintf(entry->path, sizeof(entry->path), "%s", path);

    return index_save(index);
}
```

Task 4

4A

```
commit 6b92b190cd161a391e5c2874ae2c919c17cf4914b0c355f04c2a1fd5b2f52b89
Author: PES User <pes@localhost>
Date:   1776342867

    Add farewell

commit c9d78041a2191f680752601dca25d662a06f2a68da8690187992cffb28eeda0f
Author: PES User <pes@localhost>
Date:   1776342867

    Add world

commit 924f34a1ccbe5f9efd4fb90282a46546c554dbacd5f745912ca6ad6d97a1e331
Author: PES User <pes@localhost>
Date:   1776342867

    Initial commit
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
---
ref: refs/heads/main
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % ./pes log
commit 6b92b190cd161a391e5c2874ae2c919c17cf4914b0c355f04c2a1fd5b2f52b89
Author: PES User <pes@localhost>
Date:   1776342867

    Add farewell

commit c9d78041a2191f680752601dca25d662a06f2a68da8690187992cffb28eeda0f
Author: PES User <pes@localhost>
Date:   1776342867

    Add world

commit 924f34a1ccbe5f9efd4fb90282a46546c554dbacd5f745912ca6ad6d97a1e331
Author: PES User <pes@localhost>
Date:   1776342867

    Initial commit

aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS %
```

4B

```
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6b/92b190cd161a391e5c2874ae2c919c17cf4914b0c355f04c2a1fd5b2f52b89
.pes/objects/92/4f34a1ccbe5f9efd4fb90282a46546c554dbacd5f745912ca6ad6d97a1e331
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/c9/d78041a2191f680752601dca25d662a06f2a68da8690187992cffb28eeda0f
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS %
```

4c

```
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % cat .pes/refs/heads/main && echo '---' && cat .pes/HEAD
6b92b190cd161a391e5c2874ae2c919c17cf4914b0c355f04c2a1fd5b2f52b89
---
ref: refs/heads/main
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS %
```

Full integration test

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Re
Initia
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 8c4013451bf9... Initial commit

Log after first commit:
commit 8c4013451bf9be2d8ee833a528374cee10e7ae609e95e597265ba35ac78854da
Author: PES User <pes@localhost>
Date:   1776344417

    Initial commit


--- Second Commit ---
Committed: 84997859e395... Update file.txt

--- Third Commit ---
Committed: d43a7677833a... Add farewell

--- Full History ---
commit d43a7677833ac2df00b0babc67dee570482477685baccbdee63ab9ef9dbe069a
Author: PES User <pes@localhost>
Date:   1776344417

    Add farewell

commit 84997859e395f6794e37cfbe4e8b692a6486a32242faee03d5f01b3deab895b8
Author: PES User <pes@localhost>
Date:   1776344417

    Update file.txt

commit 8c4013451bf9be2d8ee833a528374cee10e7ae609e95e597265ba35ac78854da
Author: PES User <pes@localhost>
```

Focus folder in explorer (cmd + click)

Aditya Alur (5 hours ago) Ln 30, Col 49 Spaces: 4 UTF-8 LF

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS % make test-integration
commit 8c4013451bf9be2d8ee833a528374cee10e7ae609e95e597265ba35ac78854da
Author: PES User <pes@localhost>
Date:   1776344417

    Initial commit


--- Second Commit ---
Committed: 84997859e395... Update file.txt

--- Third Commit ---
Committed: d43a7677833a... Add farewell

--- Full History ---
commit d43a7677833ac2df00b0babc67dee570482477685baccbdee63ab9ef9dbe069a
Author: PES User <pes@localhost>
Date:   1776344417

    Add farewell

commit 84997859e395f6794e37cfbe4e8b692a6486a32242faee03d5f01b3deab895b8
Author: PES User <pes@localhost>
Date:   1776344417

    Update file.txt

commit 8c4013451bf9be2d8ee833a528374cee10e7ae609e95e597265ba35ac78854da
Author: PES User <pes@localhost>
Date:   1776344417

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
d43a7677833ac2df00b0babc67dee570482477685baccbdee63ab9ef9dbe069a

--- Object Store ---
Objects created:
      10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/84/997859e395f6794e37cfbe4e8b692a6486a32242faee03d5f01b3deab895b8
.pes/objects/8c/4013451bf9be2d8ee833a528374cee10e7ae609e95e597265ba35ac78854da
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/d4/3a7677833ac2df00b0babc67dee570482477685baccbdee63ab9ef9dbe069a
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
aditya@Adityas-MacBook-Air-2 PES2UG24CS052-PES-VCS %
```

Aditya Alur (5 hours ago) Ln 30, Col 49 Spaces: 4 UTF-8 LF

Commit.c

```
// commit.c — Commit creation and history traversal
//
// Commit object format (stored as text, one field per line):
//
//   tree <64-char-hex-hash>
//   parent <64-char-hex-hash>        ← omitted for the first commit
//   author <name> <unix-timestamp>
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
//   committer <name> <unix-timestamp>
//
//   <commit message>
//
// Note: there is a blank line between the headers and the message.
//
// PROVIDED functions: commit_parse, commit_serialize, commit_walk, head_read, head_update
// TODO functions:     commit_create

#include "commit.h"
#include "index.h"
#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>

// Forward declarations (implemented in object.c)
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out);

// ——— PROVIDED ———————————————————————————————————————————————————————————————————————

// Parse raw commit data into a Commit struct.
int commit_parse(const void *data, size_t len, Commit *commit_out) {
    (void)len;
    const char *p = (const char *)data;
    char hex[HASH_HEX_SIZE + 1];

    // "tree <hex>\n"
    if (sscanf(p, "tree %64s\n", hex) != 1) return -1;
    if (hex_to_hash(hex, &commit_out->tree) != 0) return -1;
    p = strchr(p, '\n') + 1;

    // optional "parent <hex>\n"
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
    if (strncmp(p, "parent ", 7) == 0) {
        if (sscanf(p, "parent %64s\n", hex) != 1) return -1;
        if (hex_to_hash(hex, &commit_out->parent) != 0) return -1;
        commit_out->has_parent = 1;
        p = strchr(p, '\n') + 1;
    } else {
        commit_out->has_parent = 0;
    }

    // "author <name> <timestamp>\n"
    char author_buf[256];
    uint64_t ts;
    if (sscanf(p, "author %255[^\n]\n", author_buf) != 1) return -1;
    // split off trailing timestamp
    char *last_space = strrchr(author_buf, ' ');
    if (!last_space) return -1;
    ts = (uint64_t)strtoull(last_space + 1, NULL, 10);
    *last_space = '\0';
    snprintf(commit_out->author, sizeof(commit_out->author), "%s", author_buf);
    commit_out->timestamp = ts;
    p = strchr(p, '\n') + 1;  // skip author line
    p = strchr(p, '\n') + 1;  // skip committer line
    p = strchr(p, '\n') + 1;  // skip blank line

    snprintf(commit_out->message, sizeof(commit_out->message), "%s", p);
    return 0;
}

// Serialize a Commit struct to the text format.
// Caller must free(*data_out).
int commit_serialize(const Commit *commit, void **data_out, size_t *len_out) {
    char tree_hex[HASH_HEX_SIZE + 1];
    char parent_hex[HASH_HEX_SIZE + 1];
    hash_to_hex(&commit->tree, tree_hex);

    char buf[8192];
    int n = 0;
    n += snprintf(buf + n, sizeof(buf) - n, "tree %s\n", tree_hex);
    if (commit->has_parent) {
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
        hash_to_hex(&commit->parent, parent_hex);
        n += snprintf(buf + n, sizeof(buf) - n, "parent %s\n", parent_hex);
    }
    n += snprintf(buf + n, sizeof(buf) - n,
                  "author %s %" PRIu64 "\n"
                  "committer %s %" PRIu64 "\n"
                  "\n"
                  "%s",
                  commit->author, commit->timestamp,
                  commit->author, commit->timestamp,
                  commit->message);

    *data_out = malloc(n + 1);
    if (!*data_out) return -1;
    memcpy(*data_out, buf, n + 1);
    *len_out = (size_t)n;
    return 0;
}

// Walk commit history from HEAD to the root.
int commit_walk(commit_walk_fn callback, void *ctx) {
    ObjectID id;
    if (head_read(&id) != 0) return -1;

    while (1) {
        ObjectType type;
        void *raw;
        size_t raw_len;
        if (object_read(&id, &type, &raw, &raw_len) != 0) return -1;

        Commit c;
        int rc = commit_parse(raw, raw_len, &c);
        free(raw);
        if (rc != 0) return -1;

        callback(&id, &c, ctx);

        if (!c.has_parent) break;
        id = c.parent;
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
    }
    return 0;
}

// Read the current HEAD commit hash.
int head_read(ObjectID *id_out) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0'; // strip newline

    char ref_path[512];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(ref_path, sizeof(ref_path), "%s/%s", PES_DIR, line + 5);
        f = fopen(ref_path, "r");
        if (!f) return -1; // Branch exists but has no commits yet
        if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
        fclose(f);
        line[strcspn(line, "\r\n")] = '\0';
    }
    return hex_to_hash(line, id_out);
}

// Update the current branch ref to point to a new commit atomically.
int head_update(const ObjectID *new_commit) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0';

    char target_path[520];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(target_path, sizeof(target_path), "%s/%s", PES_DIR, line + 5);
    } else {
        snprintf(target_path, sizeof(target_path), "%s", HEAD_FILE); // Detached HEAD
```

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

```
    }

    char tmp_path[528];
    snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", target_path);

    f = fopen(tmp_path, "w");
    if (!f) return -1;

    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(new_commit, hex);
    fprintf(f, "%s\n", hex);

    fflush(f);
    fsync(fileno(f));
    fclose(f);

    return rename(tmp_path, target_path);
}

// ——— TODO: Implement these ———————————————————————————————————————————————

// Create a new commit from the current staging area.
//
// HINTS - Useful functions to call:
//   - tree_from_index   : writes the directory tree and gets the root hash
//   - head_read         : gets the parent commit hash (if any)
//   - pes_author        : retrieves the author name string (from pes.h)
//   - time(NULL)        : gets the current unix timestamp
//   - commit_serialize  : converts the filled Commit struct to a text buffer
//   - object_write      : saves the serialized text as OBJ_COMMIT
//   - head_update       : moves the branch pointer to your new commit
//
// Returns 0 on success, -1 on error.
int commit_create(const char *message, ObjectID *commit_id_out) {
    ObjectID tree_id;
    if (tree_from_index(&tree_id) != 0) return -1;

    Commit commit;
```

```
    memset(&commit, 0, sizeof(commit));
    commit.tree = tree_id;
    commit.has_parent = 0;

    if (head_read(&commit.parent) == 0) {
        commit.has_parent = 1;
    }

    snprintf(commit.author, sizeof(commit.author), "%s", pes_author());
    commit.timestamp = (uint64_t)time(NULL);
    snprintf(commit.message, sizeof(commit.message), "%s", message);

    void *data = NULL;
    size_t len = 0;
    if (commit_serialize(&commit, &data, &len) != 0) return -1;

    ObjectID new_commit;
    int rc = object_write(OBJ_COMMIT, data, len, &new_commit);
    free(data);
    if (rc != 0) return -1;

    if (head_update(&new_commit) != 0) return -1;

    if (commit_id_out) {
        *commit_id_out = new_commit;
    }

    return 0;
}
```

Phase 5

5.1

To implement `pes checkout <branch>`, the system must update the `.pes/HEAD` file to point to the selected branch (e.g., `ref: refs/heads/<branch>`), then read the commit hash stored in that branch file. From this commit, the corresponding tree object is loaded, which represents the full snapshot of the project. The working directory must then be updated to match this tree by deleting or replacing tracked files and recreating

the directory structure from the object store. This operation is complex because it involves reconstructing the filesystem, handling nested directories, preserving file modes, and ensuring that existing uncommitted changes are not accidentally overwritten.

5.2

To detect a dirty working directory conflict, the system compares the current working files with the index. For each tracked file, the content of the working directory file is hashed and compared with the hash stored in the index. If the hashes differ, the file has uncommitted changes. Then, the system checks whether this file would be modified by the target branch (i.e., its hash differs between the current and target commit). If both conditions are true, the checkout must be refused to prevent data loss. This ensures that local changes are not overwritten by switching branches.

5.3

In a detached HEAD state, the HEAD points directly to a commit rather than a branch reference. If new commits are made in this state, they are created normally but are not associated with any branch, making them unreachable once the user switches away. These commits can still be recovered as long as their hashes are known, either by creating a new branch pointing to them or by using tools like the reflog to locate recent commits. However, if left unreferenced, they may eventually be removed by garbage collection.

Phase 6

6.1

To find and delete unreachable objects, the algorithm starts from all root references, which include branch heads (e.g., files in `.pes/refs/heads/`) and possibly HEAD. From each of these commits, it performs a graph traversal (DFS or BFS), following commit → tree → blob relationships, and marks every visited object as "reachable." A hash-based data structure such as a **hash set** is ideal for tracking reachable object IDs efficiently, since it allows constant-time insertion and lookup while avoiding duplicates. After traversal, the system scans the `.pes/objects` directory and deletes any object whose hash is not present in the reachable set. For a repository with around 100,000 commits and 50 branches, although there are many branches, they usually share history, so the traversal will visit roughly all unique commits once, along with their associated trees and blobs. This could result in visiting on the order of a few hundred thousand objects (commits + trees + blobs), not 50 × 100,000, due to shared ancestry.

NAME – ADITYA ALUR , SECTION – A , SEMESTER – 4 , BRANCH – CSE , SRN – PES2UG24CS052 , TOPIC – Orange problem OS

6.2

Running garbage collection concurrently with a commit operation is dangerous because of race conditions between object creation and reachability marking. For example, a commit operation may first write blob and tree objects to the object store, but has not yet updated any branch reference to point to the new commit. If garbage collection runs at this moment, it may not see these new objects as reachable (since no branch points to them yet) and may delete them. Immediately after, the commit operation updates the branch to reference a commit whose underlying objects have been removed, leading to repository corruption. Git avoids this by ensuring safety through mechanisms such as object creation ordering, temporary files, and reference updates being atomic. Additionally, Git's garbage collection uses strategies like marking objects as "in use," delaying deletion, and using reflogs to retain recent objects, ensuring that newly created but not-yet-referenced objects are not prematurely deleted.